

Clase 22 - Repaso I3

Profesor: **IIC2343 - Arquitectura de Computadores**

- Felipe Valenzuela González

Correo:

frvalenzuela@alumni.uc.cl

Resumen del contenido

Arquitectura de Computadores: Mejoras y extensiones

En lo que hemos visto hasta ahora

- Una máquina programable que ejecuta programas
- Interacción con dispositivos de entrada y salida

Lo que nos queda ver **mejoras**:

- Mejora los accesos a memoria ✓
- Mejora en lo que respecta a la ejecución de más de un programa ✓
- Mejora en lo que respecta a la ejecución de instrucciones en la CPU ✓

Jerarquía de Memoria

Generalmente, el código que escribimos cumple dos principios de localidad

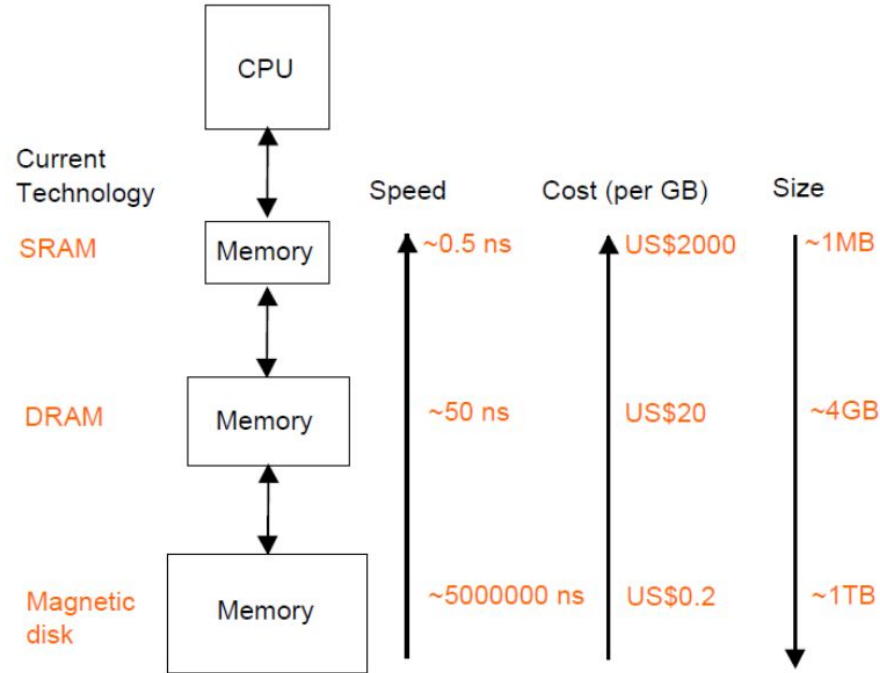
1. Principio de localidad temporal

Es probable que un dato obtenido de memoria sea usado posteriormente en otra operación

2. Principio de localidad espacial

Es probable que datos cercanos al buscado también sean usados (bloque)

Jerarquía de Memoria es una buena opción para mejorar el rendimiento



¿Dudas?

Memoria Caché

Evaluación Tiempo promedio

Tiempo promedio en caso de *hit*

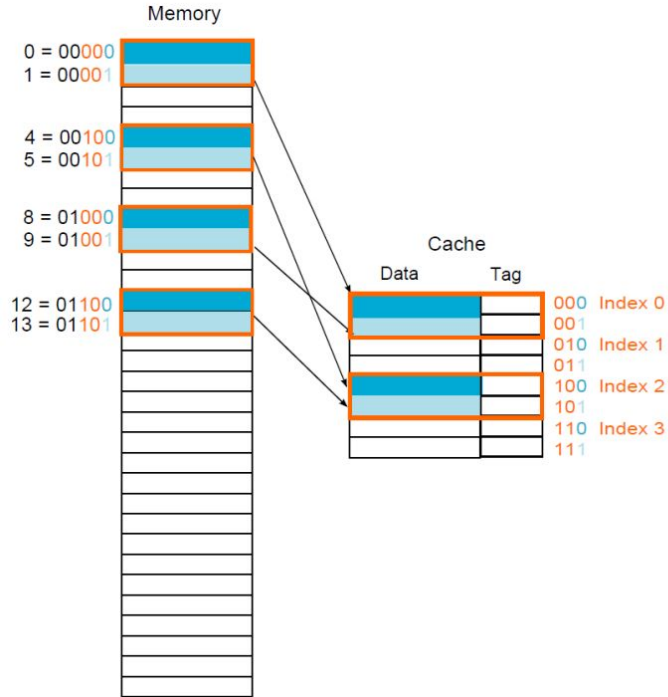
Tiempo promedio en caso de *miss* *

$$\begin{aligned} TP &= HR * HT + (1 - HR) * (HT + MP) \\ &= HT + (1 - HR) * MP \end{aligned}$$

Mapeo directo (directly mapped) es la función de correspondencia más simple

Revisemos un ejemplo con los siguientes datos:

- Memoria principal: 32 bytes (32 palabras)
- Caché de 8 bytes y 4 líneas



Funciones de correspondencia - *Fully associative*

- En este esquema, cada bloque de memoria puede asociarse a **cualquier línea de caché**
- Aprovecha todo el espacio y maximiza *hit-rate*
- Como se puede ocupar cualquier línea, la composición del tag para identificar si un dato se encuentra en caché se **complejiza**

Funciones de correspondencia - *N way associative*

En este esquema, también llamado **Set associative**, cada bloque de memoria puede asociarse a un conjunto de líneas de la caché.

- Divide a la caché en conjuntos de **N líneas !!No son N conjuntos!!**
- Lo mejor de los dos mundos: Cada bloque posee un mapeo directo a un conjunto y dentro de un conjunto, un bloque se asocia a cualquier línea

Políticas de reemplazos

- **Bélády**: El bloque que se utilizará más lejos en el futuro se saca. Óptimo no alcanzable en la práctica.
- **First-in First-out (FIFO)**: El primer bloque en entrar es el primero en salir. Simple, pero tonto.
- **Least Frequently Used (LFU)**: El bloque con menos accesos se saca. Mejor que FIFO, solo un poco más complejo.
- **Least Recently Used (LRU)**: El bloque con mayor tiempo sin accesos se saca. Complejo, requiere timestamp. En general el de mejor rendimiento.
- **Random**: Muy rápido y con rendimiento algo inferior a LFU y LRU.

Políticas de escritura

- **Write-through:** el bloque modificado es escrito inmediatamente en la memoria principal
- **Write-back:** el bloque modificado es escrito en la memoria principal sólo cuando va a ser sustituido

¿Dudas?

Coherencia de Caché

Tipos de arquitectura paralela - Memoria compartida

- Una política de escritura **write-through**, no habrá problemas ya que la memoria será consistente para todos los procesadores (más tiempo)
- Actualmente se opta por velocidad por tanto hay se sigue el protocolo de escritura **write-back**, pero de este nace un nuevo problema: la **coherencia de caché**

Coherencia de caché - Protocolo MSI

- **Modified:** Indica que el contenido de la línea fue modificado y no es consistente con el bloque de la memoria principal. Este puede existir solo en una caché para un mismo bloque compartido.
- **Shared:** Indica que el contenido de la línea se encuentra en al menos una memoria caché, pero no se ha modificado.
- **Invalid:** Indica que el contenido de la línea es inválido, que puede ser por dato no existente o por solicitud desde el bus compartido.

Protocolo MSI - Diagrama General

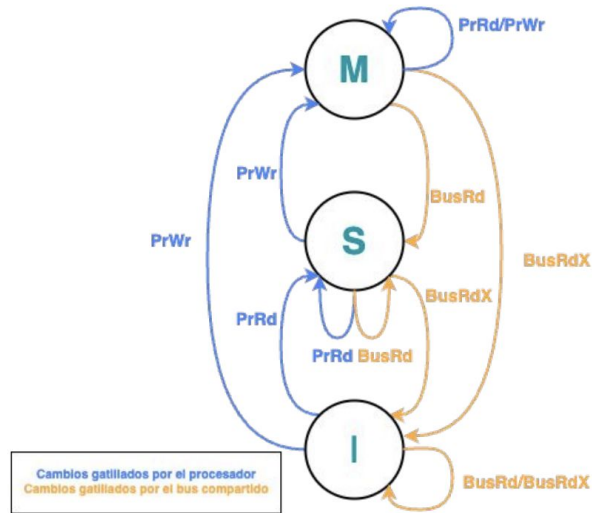


Diagrama de estado de una línea de caché para el protocolo MSI.

Protocolo MESI - Diagramas

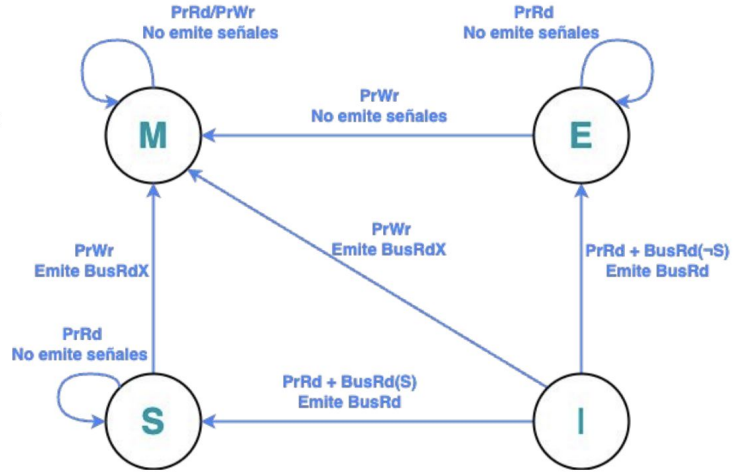


Diagrama de estado de una línea de caché para el protocolo MESI y eventos del procesador.

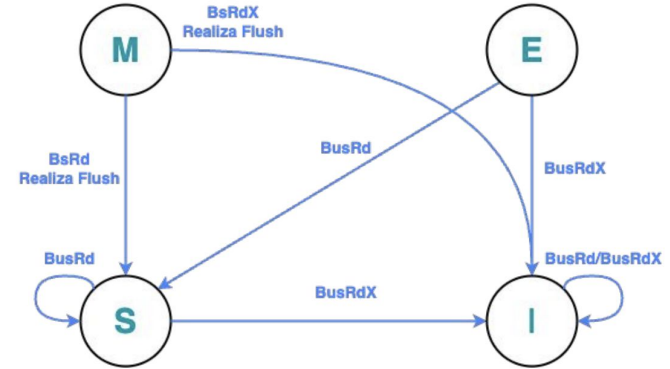


Diagrama de estado de una línea de caché para el protocolo MESI y eventos del bus compartido.

Ejercicio típico

[illegible]

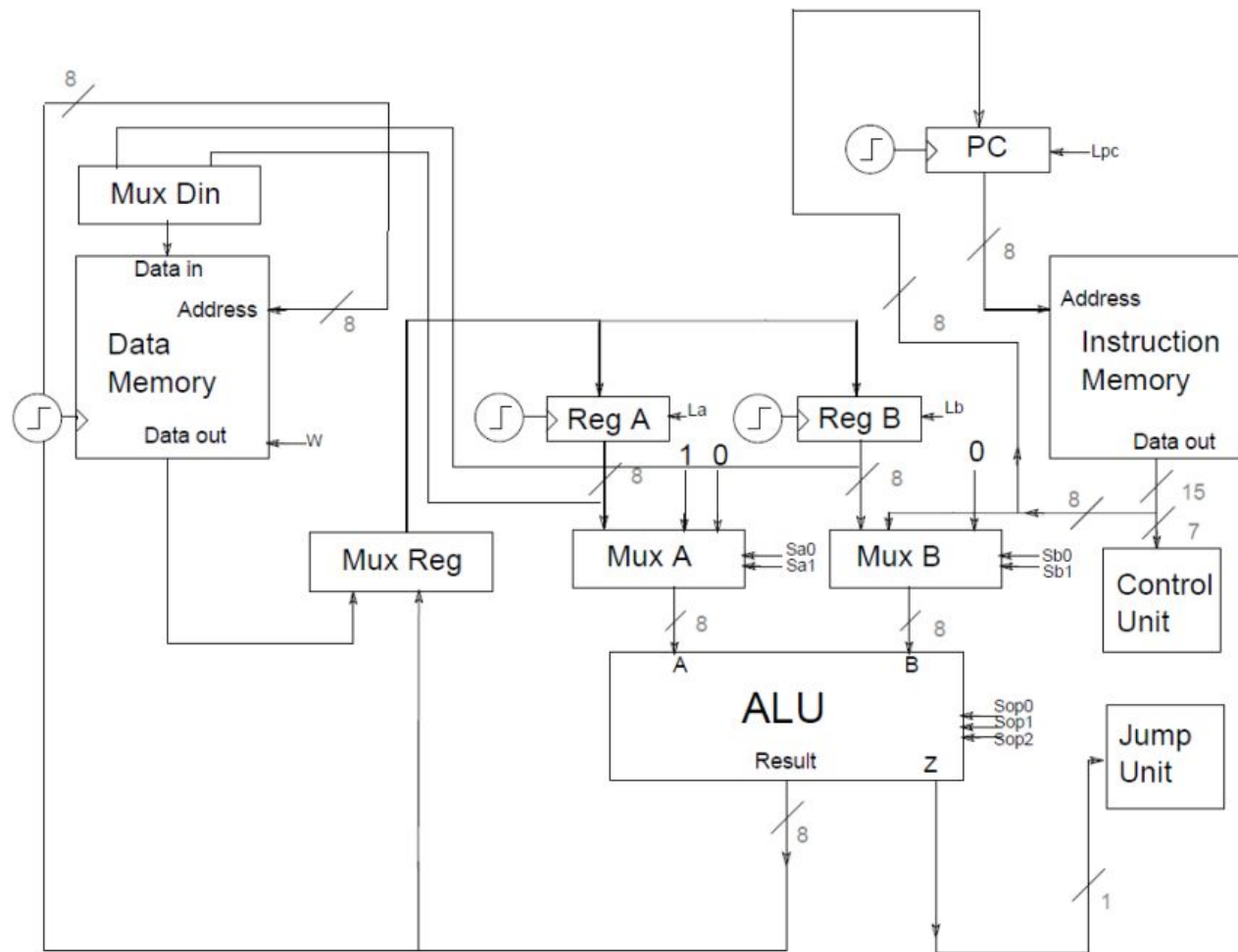
¿Dudas?

Paralelismo

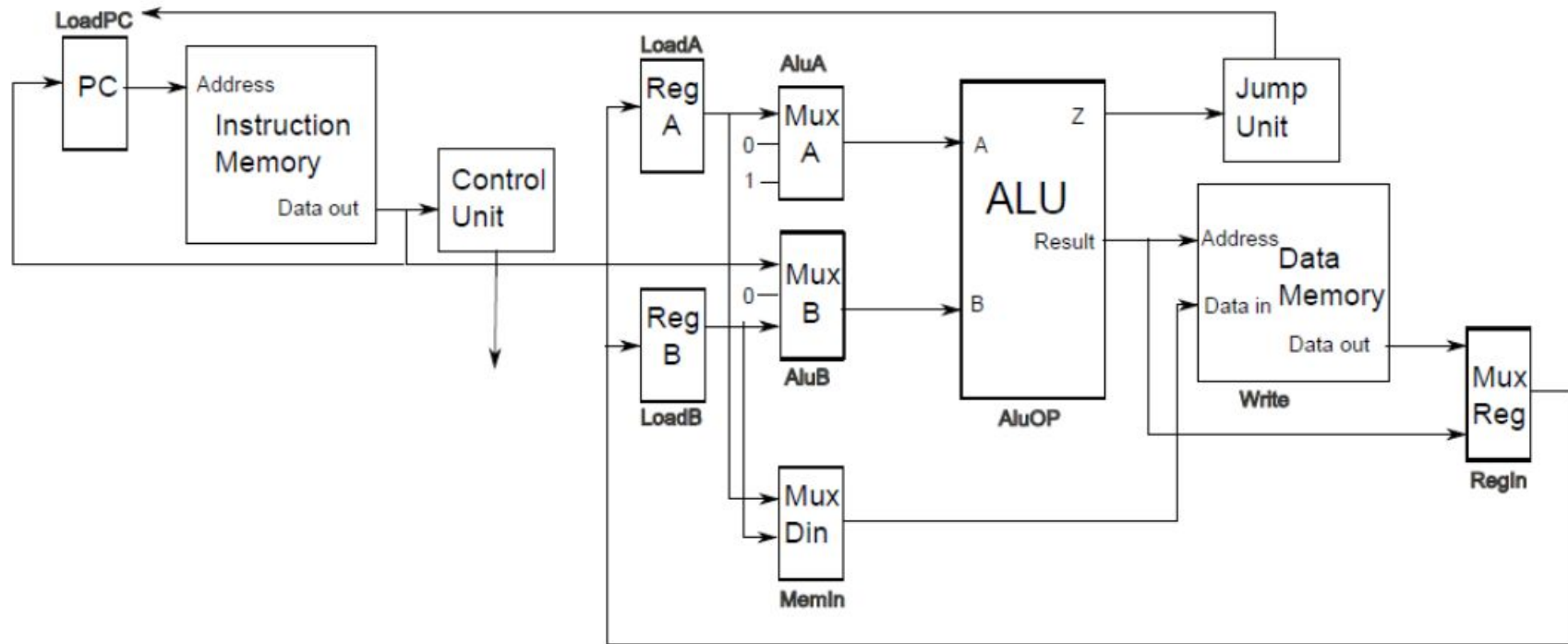
Flujo de instrucción: General

- 1- Lectura de instrucción desde memoria (**Fetch**)
- 2-Decodificar instr. en unidad de control (**Decode**)
- 3-Ejecutar en ALU (**Execute**)
- 4- Acceder a memoria (**Mem**)
- 5- Escribir resultado en registro (**Write Back**)

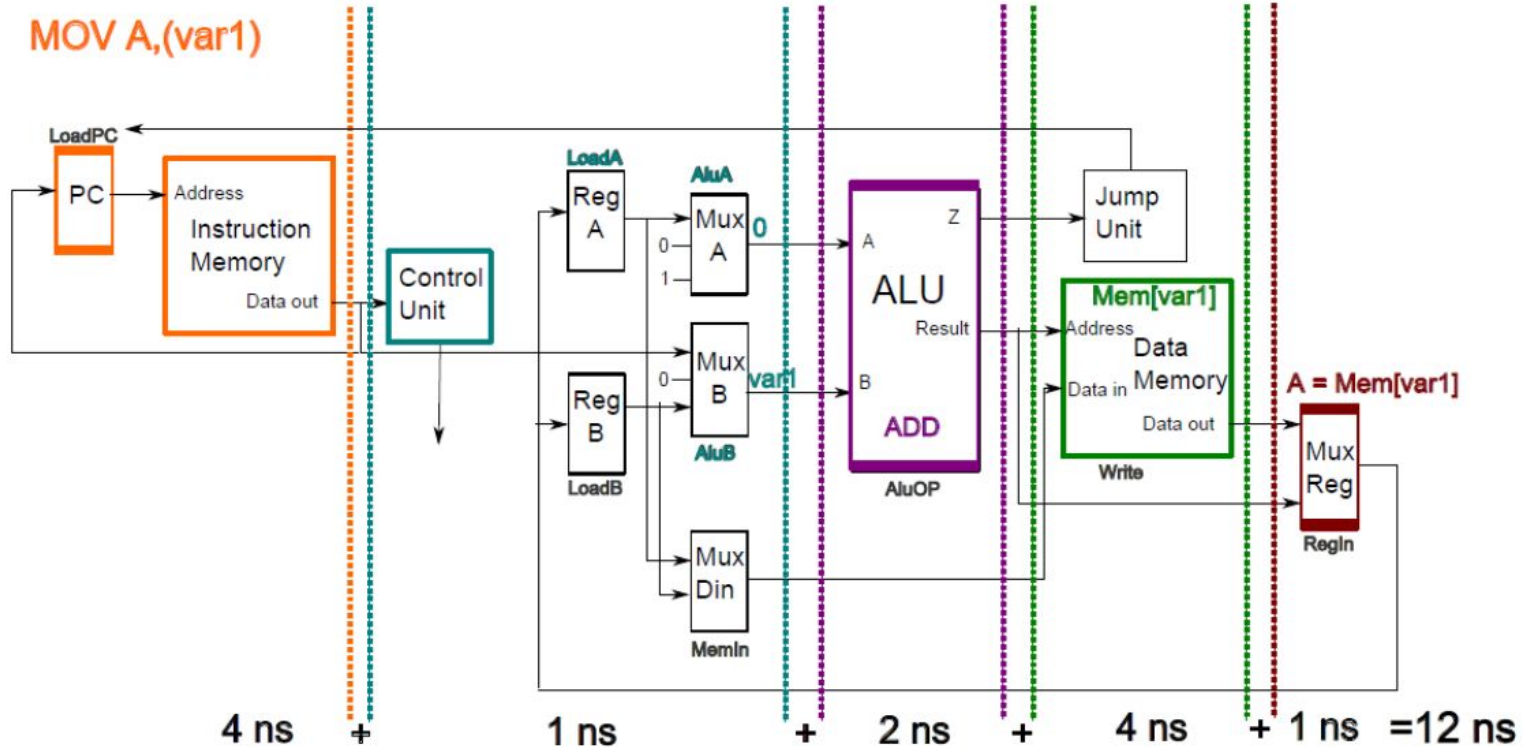
Nuevo Computador



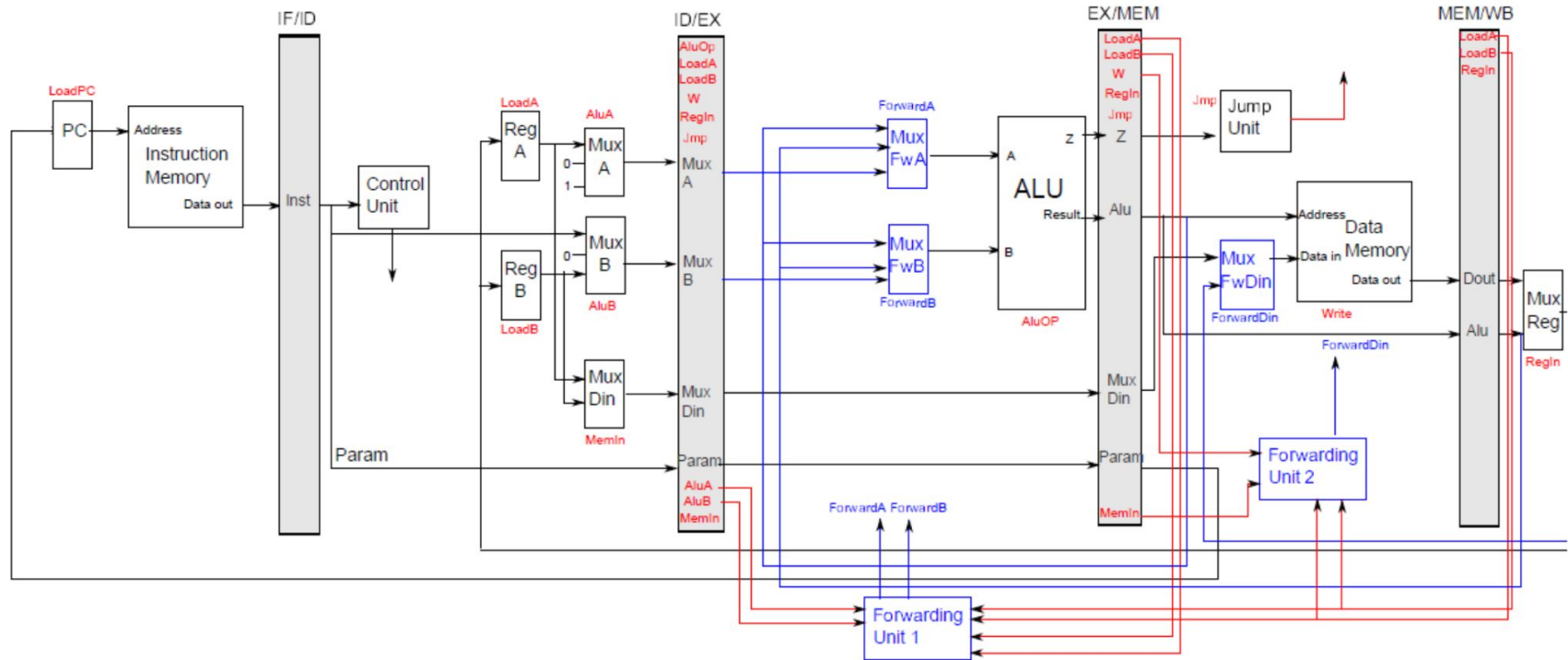
Nuevo Computador Re-organizado



Nuevo Computador Re-organizado: Tiempo MOV A,(var)

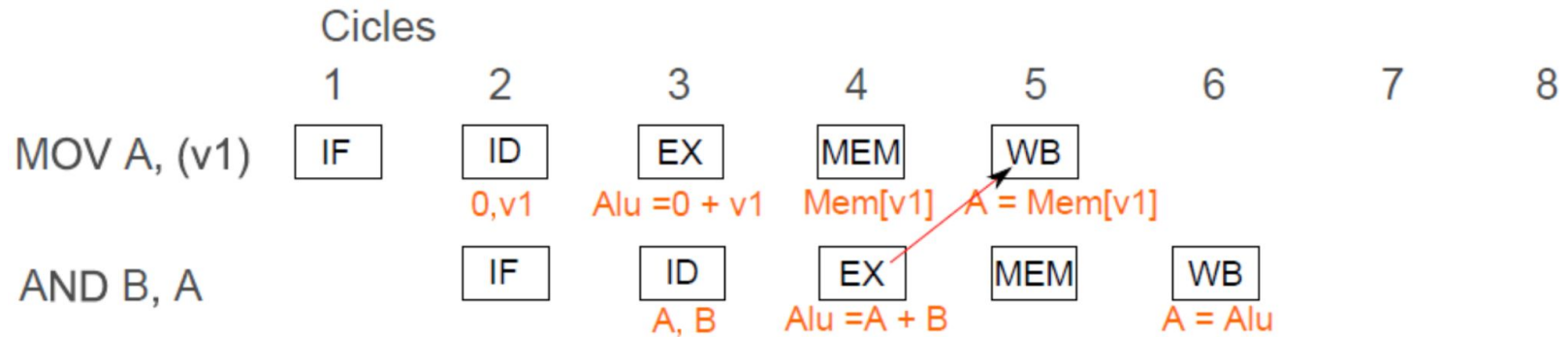


Computador básico con pipeline: Hazards de datos



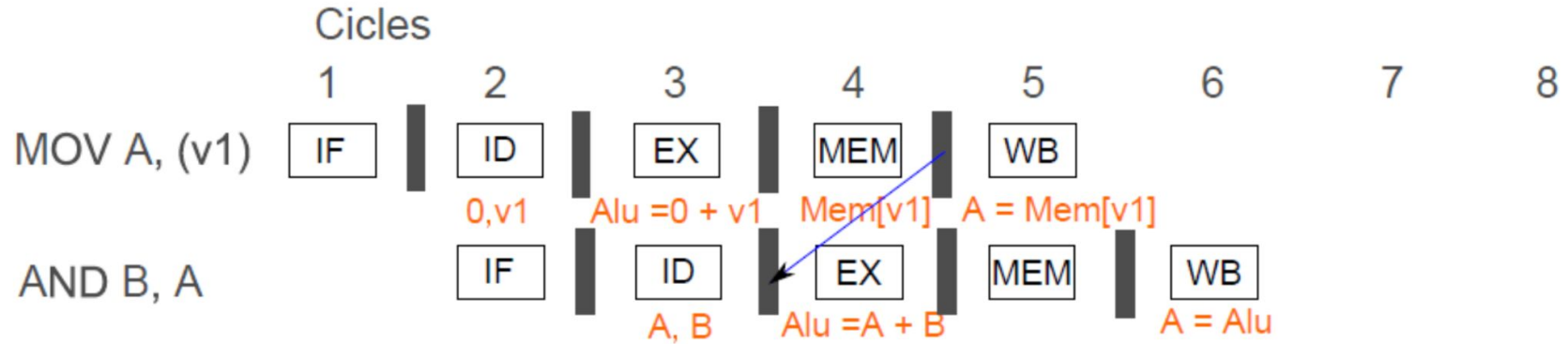
Computador básico con pipeline: Hazards de datos

- ¿Qué hacemos en este caso ?



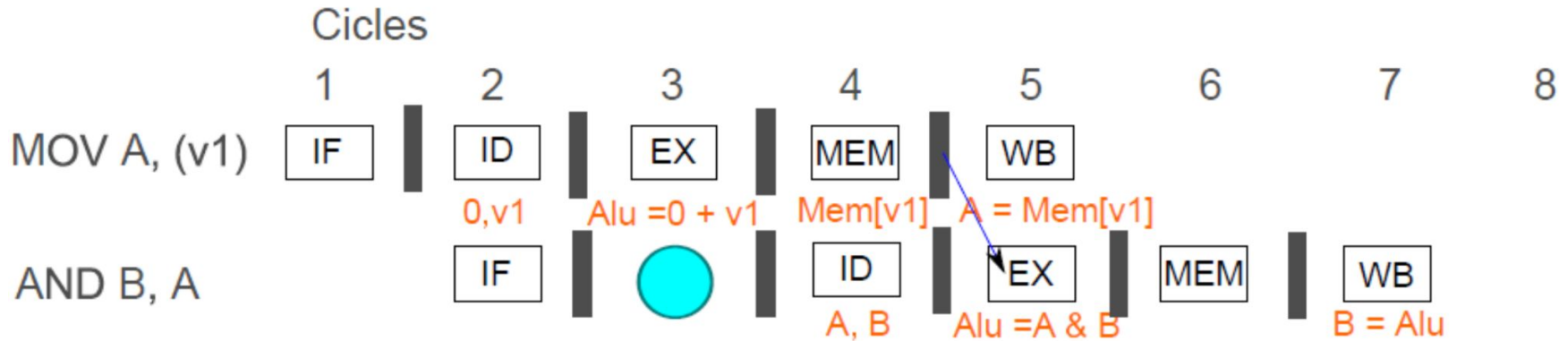
Computador básico con pipeline: Hazards de datos

- **Forwarding** llega muy tarde, **no** es una solución factible

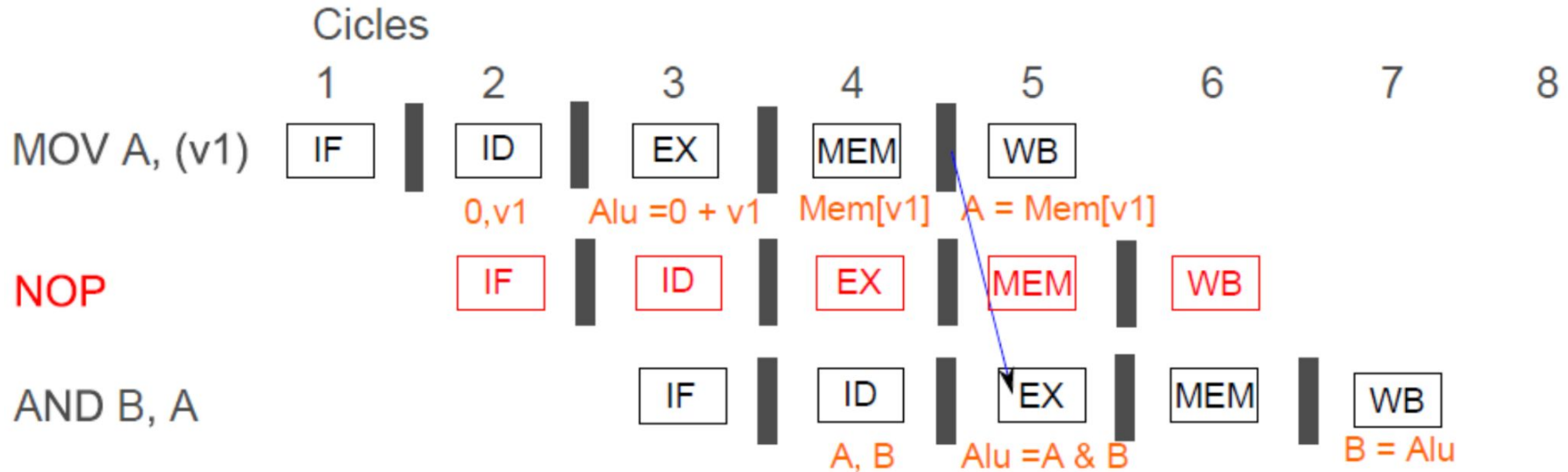


Computador básico con pipeline: Hazards de datos

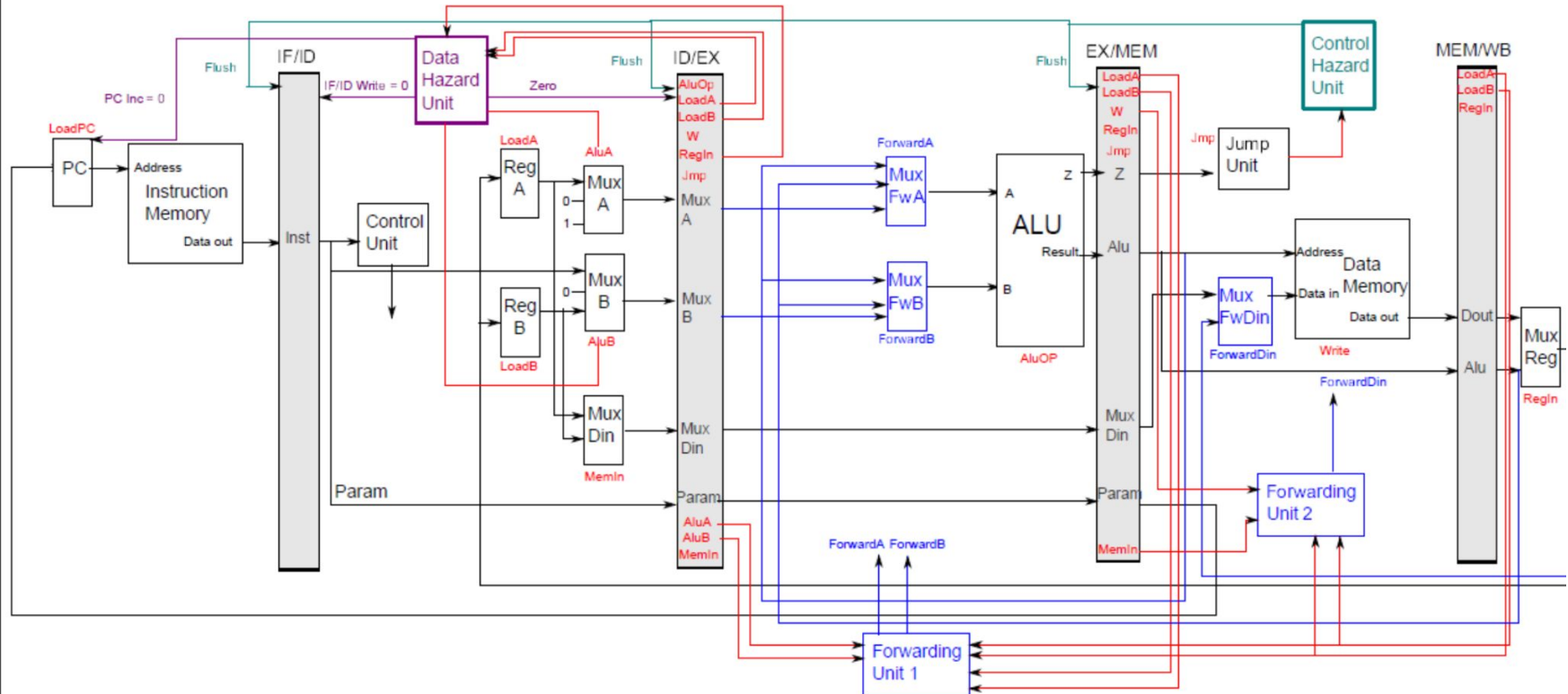
- Necesitamos que el procesador espere, lo que se conoce como **stalling**



Computador básico con pipeline: Stalling por software



Computador básico con pipeline: Hazards de control



Computador básico con pipeline: Hazards estructurales

- Estos ocurren cuando ocurren cuando dos etapas necesitan usar la misma unidad funcional
- **Ejemplo:**

Etapas IF y MEM en una arquitectura Von Neumann. Como en este tipo de arquitectura la memoria de datos e instrucciones se encuentran en una única unidad, estas dos etapas requieren acceder a ella en un mismo instante de tiempo.

- ¿Cómo se podría resolver este caso?

Computador básico con pipeline: Hazards estructurales

- **Solución 1:** Agregar stalling de forma intercalada a cada etapa para que haya un solo acceso por ciclo.

Problema: Pérdida de 2 ciclos por instrucción, siempre.

- **Solución 2:** Agregar otra unidad de memoria.

Problema: Aumento significativo de costo, cambio estructural de la arquitectura.

Computador básico con pipeline: Hazards estructurales

Solución real:

Hacer uso de una **caché split**. ¿Por qué esto resuelve el problema?

- Las **caché split** tienen la particularidad de permitir dos accesos de memoria concurrentes: uno para la sección de datos y otra para la sección de instrucciones
- Así, el **pipeline** trata los accesos de memoria como una arquitectura **Harvard**, aunque sea **Von Neumann**

¿Dudas?

Clase 22 - Repaso I3

Profesor: **IIC2343 - Arquitectura de Computadores**
- Felipe Valenzuela González
Correo:
frvalenzuela@alumni.uc.cl